**Развертывание веб-сайта с помощью Terraform**

1. Создаю директорию для проекта terraform-project-hw42. Внутри создаю следующие файлы main.tf (основная конфигурация), variables.tf (переменные), terraform.tfvars (для указания значения переменных) и outputs.tf (вывод информации о созданных ресурсах)
2. До этого создаю SSH ключ, чтобы использовать в дальнейшей конфигурации по пути /home/anastasia/.ssh/id_rsa.pub для подключения к ВМ, которая будет создана
3. Итак, создаю сначала файл конфигурации main.tf со следующим содержимым. Описываем сеть, подсеть, группу безопасности, ВМ и SSH ключи для доступа к созданной ВМ, с использованием переменных (var.). Также разрешаем траффик, чтобы иметь доступ к сайту из внешки и получать необходимые обновления и возможности установки компонентов, например, nginx:

```
terraform {
  required_providers {
    yandex = {
      source = "yandex-cloud/yandex"
    }
  }
  required_version = ">= 0.13"
}

provider "yandex" {
  zone = var.zone
  folder_id = var.folder_id
  cloud_id  = var.cloud_id
}

resource "yandex_vpc_network" "web_network" {
  name = var.network_name
}

resource "yandex_vpc_subnet" "web_subnet" {
  name           = var.subnet_name
  zone           = var.zone
  network_id     = yandex_vpc_network.web_network.id
  v4_cidr_blocks = var.subnet_cidr
}

resource "yandex_vpc_security_group" "web_sg" {
  name        = var.sg_name
  description = "Security group for web server"
  network_id  = yandex_vpc_network.web_network.id
```

```
  # Разрешить входящий HTTP трафик (порт 80)
  ingress {
    protocol       = "TCP"
    v4_cidr_blocks = ["0.0.0.0/0"]
    port           = 80
  }

  # Разрешить входящий HTTPS трафик (порт 443)
  ingress {
    protocol       = "TCP"
    v4_cidr_blocks = ["0.0.0.0/0"]
    port           = 443
  }

  # Разрешить SSH доступ (порт 22)
  ingress {
    protocol       = "TCP"
    v4_cidr_blocks = ["0.0.0.0/0"]
    port           = 22
  }

  # Разрешить ICMP (ping)
  ingress {
    protocol       = "ICMP"
    v4_cidr_blocks = ["0.0.0.0/0"]
  }

  # Разрешить весь исходящий трафик
  egress {
    protocol       = "ANY"
    v4_cidr_blocks = ["0.0.0.0/0"]
    from_port      = 0
    to_port        = 65535
  }
}
```

```
resource "yandex_compute_disk" "web_disk" {
  name     = var.disk_name
  type     = var.disk_type
  zone     = var.zone
  size     = var.disk_size
}

resource "yandex_compute_instance" "web_server" {
  name        = var.instance_name
  platform_id = "standard-v3"
  description = var.instance_description
  hostname    = var.hostname
  zone        = var.zone

  resources {
    cores         = var.cores
    core_fraction = var.core_fraction
    memory        = var.memory
  }

  boot_disk {
    disk_id = yandex_compute_disk.web_disk.id
  }

  network_interface {
    subnet_id          = yandex_vpc_subnet.web_subnet.id
    nat                = var.assign_public_ip
    security_group_ids = [yandex_vpc_security_group.web_sg.id]
  }
```

```
  metadata = {
    ssh-keys  = "${var.vm_username}:${file(var.ssh_public_key_path)}"
    user-data = <<-EOF
              #cloud-config
              package_update: true
              packages:
                - nginx
              runcmd:
                - systemctl start nginx
                - systemctl enable nginx
                - echo "<h1>Добро пожаловать на сервер!</h1>" > /var/www/html/index.nginx-debian.html
              EOF
  }

  labels = {
    environment = var.environment
  }
}
```

4. Далее создаю файл variables.tf, в котором описываю переменные, которые в свою очередь используются в главном файле конфигурации:

```
variable "zone" {
  description = "Зона доступности"
  default     = "ru-central1-b"
}

variable "cloud_id" {
  description = "Идентификатор облака"
  default     = "b1gld4v1m4t195n0ock6"
}

variable "folder_id" {
  description = "Идентификатор папки"
  default     = "b1gpmga0r88fkk7tbv6m"
}

variable "network_name" {
  type        = string
  description = "Имя сети"
  default     = "web-network"
}

variable "subnet_name" {
  type        = string
  description = "Имя подсети"
  default     = "web-subnet"
}

variable "subnet_cidr" {
  type        = list(string)
  default     = ["192.168.10.0/24"]
}
```

```
variable "sg_name" {
  type        = string
  description = "Имя группы безопасности"
  default     = "web-security-group"
}

variable "disk_name" {
  type        = string
  description = "Имя диска"
  default     = "web-server-disk"
}

variable "disk_type" {
  type    = string
  default = "network-ssd"
}

variable "disk_size" {
  type        = number
  description = "Размер диска в GB"
  default     = 10
}

variable "image_id" {
  type        = string
  description = "ID образа Ubuntu 22.04 LTS"
  default     = "fd806c8slu9j1pa87msc"
}

variable "instance_name" {
  type    = string
  default = "web-server"
}
```

```
variable "instance_description" {
  type        = string
  default     = "Web server created with Terraform"
}

variable "hostname" {
  type        = string
  description = "Hostname"
  default     = "web-server"
}

variable "cores" {
  type        = number
  description = "Количество CPU"
  default     = 2
}

variable "core_fraction" {
  type        = number
  default     = 50
}

variable "memory" {
  type        = number
  description = "RAM in GB"
  default     = 2
}

variable "assign_public_ip" {
  type        = bool
  description = "Публичный IP address"
  default     = true
}
```

```
variable "vm_username" {
  type        = string
  description = "Имя пользователя для подключения к ВМ"
  default     = "puppy"
}

variable "ssh_public_key_path" {
  type        = string
  description = "Путь до файла SSH public key"
  default     = "/home/anastasia/.ssh/id_rsa.pub"
}

variable "environment" {
  type        = string
  description = "Наименование окружения (dev, staging, prod)"
  default     = "dev"
}
```

5. Далее создаю файл terraform.tfvars, в котором описываются конкретные значения для переменных из файла variables.tf

```
zone      = "ru-central1-b"
cloud_id  = "b1gld4v1m4t195n0ock6"
folder_id = "b1gpmga0r88fkk7tbv6m"

network_name = "web-network"
subnet_name  = "web-subnet"
subnet_cidr  = ["192.168.10.0/24"]

sg_name = "web-security-group"

disk_name = "web-server-disk"
disk_type = "network-ssd"
disk_size = 10
image_id  = "fd806c8slu9j1pa87msc"

instance_name        = "web-server"
instance_description = "Production web server"
hostname             = "web-server"
cores                = 2
core_fraction        = 50
memory               = 2
assign_public_ip     = true

ssh_public_key_path = "/home/anastasia/.ssh/id_rsa.pub"
vm_username         = "puppy"
environment = "production"
```

Image_id обязателен, иначе он не создаст ОС, а просто пустую ВМ. Посмотреть актуальные версии можно yc compute image list --folder-id standard-images --format json | \
  jq -r '.[] | select(.family=="ubuntu-2204-lts") | "\(.id) \(.name)"'

```
fd806c8slu9j1pa87msc ubuntu-22-04-lts-v20260615
fd80bm0rh4rkepi5ksdi ubuntu-22-04-lts-v20230925
fd80ok8sil1fn2gqbm6h ubuntu-22-04-lts-v20241014
fd8136cthbfr503tjj72 ubuntu-22-04-lts-v20250505
fd817i7o8012578061ra ubuntu-22-04-lts-v20250623
fd81no7ub0p1nooono37 ubuntu-22-04-lts-v20241111
fd81r1dpns2m4mgssm0q ubuntu-22-04-lts-v20250303
fd81radk00nmm2jpqh94 ubuntu-22-04-lts-v20251229
fd828qsqv1jtq2qec4m5 ubuntu-22-04-lts-v20240610
fd82nvvtllmimo92uoul ubuntu-22-04-lts-v20230911
fd82sqrj4uk9j7vlki3q ubuntu-22-04-lts-v20230424
fd82vchjp2kdjiuam29k ubuntu-22-04-lts-v20240422
fd8308aanqma9v5n76aj ubuntu-22-04-lts-v20250127
fd830gae25ve4glajdsj ubuntu-22-04-lts-v20230918
fd833v6c5tb0udvk4jo6 ubuntu-22-04-lts-v20240325
fd837n64isqsu1o0rqfn ubuntu-22-04-lts-v20250616
fd83m7rp3r4l12c2keph ubuntu-22-04-lts-v20240624
```

6. Далее создаю файл outputs.tf, в котором описывается какой вывод будет отображен после выполнения создания конфигурации:

```
output "web_server_name" {
  value       = yandex_compute_instance.web_server.name
  description = "Name of the web server"
}

output "web_server_internal_ip" {
  value       = yandex_compute_instance.web_server.network_interface.0.ip_address
  description = "Internal IP address of the web server"
}

output "web_server_external_ip" {
  value       = yandex_compute_instance.web_server.network_interface.0.nat_ip_address
  description = "External IP address of the web server"
}

output "web_server_fqdn" {
  value       = yandex_compute_instance.web_server.fqdn
  description = "FQDN of the web server"
}

output "ssh_connection_command" {
  value       = "ssh -i ${var.ssh_public_key_path} ${var.vm_username}@${yandex_compute_instance.web_server.network_interface.0.nat_ip_address}"
  description = "Команда для подключения к ВМ по SSH"
}

output "vm_username" {
  value       = var.vm_username
  description = "Имя пользователя для подключения к ВМ"
}
```

7. Теперь проверяем конфигурацию с помощью terraform validate:

```
anastasia@compute-vm:~/terraform-project-hw42$ terraform validate
Success! The configuration is valid.
```

8. Просматриваем командой terraform plan план выполнения настройки по конфигурации:

```
anastasia@compute-vm:~/terraform-project-hw42$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # yandex_compute_disk.web_disk will be created
  + resource "yandex_compute_disk" "web_disk" {
      + block_size  = 4096
      + created_at  = (known after apply)
      + folder_id   = (known after apply)
      + id          = (known after apply)
      + image_id    = "f2es2ed0ju8m8ukho9ju"
      + name        = "web-server-disk"
      + product_ids = (known after apply)
      + size        = 10
      + status      = (known after apply)
      + type        = "network-ssd"
      + zone        = "ru-central1-b"

      + disk_placement_policy (known after apply)

      + hardware_generation (known after apply)
    }

  # yandex_compute_instance.web_server will be created
  + resource "yandex_compute_instance" "web_server" {
      + created_at                = (known after apply)
      + description               = "Production web server"
      + folder_id                 = (known after apply)
      + fqdn                      = (known after apply)
      + gpu_cluster_id            = (known after apply)
      + hardware_generation       = (known after apply)
      + hostname                  = "web-server"
      + id                        = (known after apply)
      + labels                    = {
          + "environment" = "production"
```

```
      }
    + maintenance_grace_period  = (known after apply)
    + maintenance_policy        = (known after apply)
    + metadata                  = {
        + "ssh-keys"  = <<-EOT
            ubuntu:ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAACAQCbpD0hV9zp7MPpXnYZ1hSbW8KUShpxApZbWzp76CYmmM5idWlwo5OnIOm3eBQAsMcNo5GZtPXFGpTZkGxnZe8diT5nRHL1qSkCN92fDE8M4hMUNVFfRNz3mtIJs86db8I+Is/
AorF2KCGXntqdoHw4zUKldr0slkNtVuQuxhL/9ZiiXs9OMOumRQqJhhsYSBnYr21O1POEPE8yBFIn15YyeNC59Ubp/y5TAmK0PxAgMJcn+Z6UgvJXfcHelY19moHyoXcbydbs2ZPYs+iXA5FByN1gDD2pu6KqhhNDS7GARlsoyDUdHfkT++8baTss0Z4317wnX
7lbuC/8tQoOm67po+muGvYSbolrLDQp1zQKPpLgSlZe5+3N78Xkl9w8BE4qSJ0zxAOke6icxiQ0IJi3aPhuExDmteDuZ6EdbbF6nghdwocnIK+A0k/5VDowUBHFQoK7dEtkvVrRMxvAIjw2xbxjOJXU40lEDw8IIgr/VN+D51y8+J5oOVQVBAhn35Wf+nhEqb0
e6cWJ9ZGp9TPq1ViQ6m5zA26xg+6JQUyWlKanr2YvM6+RPp+EAWASg5rbtI5yS9sp7U/34YvChbffeq4yYNW9Ps4HnvyjJsoNENAEdgbkhzpzZgAiWMX7JkHhenN+nM9mE1AoCvdk0oEaoDL32XEWDRLgMQ5LxowBGw== anastasia@compute-vm
          EOT
        + "user-data" = <<-EOT
            package_update: true
            packages:
              - nginx
            runcmd:
              - systemctl start nginx
              - systemctl enable nginx
              - echo "<h1>Добро пожаловать на сервер!</h1>" > /var/www/html/index.nginx-debian.html
          EOT
      }
    + name                      = "web-server"
    + network_acceleration_type = "standard"
    + platform_id               = "standard-v1"
    + reserved_instance_pool_id = (known after apply)
    + status                    = (known after apply)
    + zone                      = "ru-central1-b"

    + boot_disk {
        + auto_delete = true
        + device_name = (known after apply)
        + disk_id     = (known after apply)
        + mode        = (known after apply)

        + initialize_params (known after apply)
      }

    + metadata_options (known after apply)

    + network_interface {
        + index              = (known after apply)
        + ip_address         = (known after apply)
        + ipv4               = true
        + ipv6               = (known after apply)
        + ipv6_address       = (known after apply)
        + mac_address        = (known after apply)
        + nat                = true
        + nat_ip_address     = (known after apply)
        + nat_ip_version     = (known after apply)
        + security_group_ids = (known after apply)
        + subnet_id          = (known after apply)
      }

    + placement_policy (known after apply)

    + resources {
        + core_fraction = 50
        + cores         = 2
        + memory        = 2
      }

    + scheduling_policy (known after apply)
  }

# yandex_vpc_network.web_network will be created
+ resource "yandex_vpc_network" "web_network" {
    + created_at                = (known after apply)
    + default_security_group_id = (known after apply)
    + folder_id                 = (known after apply)
    + id                        = (known after apply)
    + labels                    = (known after apply)
    + name                      = "web-network"
    + subnet_ids                = (known after apply)
  }

# yandex_vpc_security_group.web_sg will be created
+ resource "yandex_vpc_security_group" "web_sg" {
    + created_at  = (known after apply)
    + description = "Security group for web server"
```

```
+ folder_id   = (known after apply)
+ id          = (known after apply)
+ labels      = (known after apply)
+ name        = "web-security-group"
+ network_id  = (known after apply)
+ status      = (known after apply)

+ egress {
    + from_port         = 0
    + id                = (known after apply)
    + labels            = (known after apply)
    + port              = -1
    + protocol          = "ANY"
    + to_port           = 65535
    + v4_cidr_blocks    = [
        + "0.0.0.0/0",
      ]
    + v6_cidr_blocks    = []
      # (3 unchanged attributes hidden)
  }

+ ingress {
    + from_port         = -1
    + id                = (known after apply)
    + labels            = (known after apply)
    + port              = -1
    + protocol          = "ICMP"
    + to_port           = -1
    + v4_cidr_blocks    = [
        + "0.0.0.0/0",
      ]
    + v6_cidr_blocks    = []
      # (3 unchanged attributes hidden)
  }
+ ingress {
    + from_port         = -1
    + id                = (known after apply)
    + labels            = (known after apply)
    + port              = 22
```

```
      + protocol          = "TCP"
      + to_port           = -1
      + v4_cidr_blocks    = [
          + "0.0.0.0/0",
        ]
      + v6_cidr_blocks    = []
        # (3 unchanged attributes hidden)
    }
  + ingress {
      + from_port         = -1
      + id                = (known after apply)
      + labels            = (known after apply)
      + port              = 443
      + protocol          = "TCP"
      + to_port           = -1
      + v4_cidr_blocks    = [
          + "0.0.0.0/0",
        ]
      + v6_cidr_blocks    = []
        # (3 unchanged attributes hidden)
    }
  + ingress {
      + from_port         = -1
      + id                = (known after apply)
      + labels            = (known after apply)
      + port              = 80
      + protocol          = "TCP"
      + to_port           = -1
      + v4_cidr_blocks    = [
          + "0.0.0.0/0",
        ]
      + v6_cidr_blocks    = []
        # (3 unchanged attributes hidden)
    }
}

# yandex_vpc_subnet.web_subnet will be created
+ resource "yandex_vpc_subnet" "web_subnet" {
    + created_at     = (known after apply)
```

```
    + folder_id      = (known after apply)
    + id             = (known after apply)
    + labels         = (known after apply)
    + name           = "web-subnet"
    + network_id     = (known after apply)
    + v4_cidr_blocks = [
        + "192.168.10.0/24",
      ]
    + v6_cidr_blocks = (known after apply)
    + zone           = "ru-central1-b"
  }

Plan: 5 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + web_server_external_ip = (known after apply)
  + web_server_fqdn        = (known after apply)
  + web_server_internal_ip = (known after apply)
  + web_server_name        = "web-server"
```

9. Как видно из пункта с проверкой конфигурации ошибок нет, поэтому теперь запускаем полноценную сборку конфигурации terraform apply:

```
yandex_compute_instance.web_server: Creating...
yandex_compute_instance.web_server: Still creating... [00m10s elapsed]
yandex_compute_instance.web_server: Still creating... [00m20s elapsed]
yandex_compute_instance.web_server: Still creating... [00m30s elapsed]
yandex_compute_instance.web_server: Creation complete after 38s [id=epdrf4lj5ieq6v28a8jf]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

web_server_external_ip = "84.252.136.104"
web_server_fqdn = "web-server.ru-central1.internal"
web_server_internal_ip = "192.168.10.21"
web_server_name = "web-server"
```

10. Теперь проверим что сервер работает, проверим с помощью http:// 84.201.140.62:

```
anastasia@compute-vm:~/terraform-project-hw42$ curl http://84.201.140.62
<h1>Добро пожаловать на сервер!</h1>
```

11. Для очистки всего созданного из конфигурации используется команды terraform destroy:

```
yandex_compute_instance.web_server: Destroying... [id=epd84b1n9i3v38g734rt]
yandex_compute_instance.web_server: Still destroying... [id=epd84b1n9i3v38g734rt, 00m10s elapsed]
yandex_compute_instance.web_server: Still destroying... [id=epd84b1n9i3v38g734rt, 00m20s elapsed]
yandex_compute_instance.web_server: Still destroying... [id=epd84b1n9i3v38g734rt, 00m30s elapsed]
yandex_compute_instance.web_server: Destruction complete after 30s
yandex_vpc_subnet.web_subnet: Destroying... [id=e2lq92q5v4jekqa6j0rh]
yandex_vpc_security_group.web_sg: Destroying... [id=enpe31kdgdlpiui48j83]
yandex_compute_disk.web_disk: Destroying... [id=epdgju3404bji5ikh2aj]
yandex_compute_disk.web_disk: Destruction complete after 0s
yandex_vpc_security_group.web_sg: Destruction complete after 0s
yandex_vpc_subnet.web_subnet: Destruction complete after 6s
yandex_vpc_network.web_network: Destroying... [id=enpbeo4nt3jq72t36mi7]
yandex_vpc_network.web_network: Destruction complete after 1s

Destroy complete! Resources: 5 destroyed.
```